

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Pattern Recognition and Text Processing

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Pattern Recognition and Text Processing
Weightage:	40% of CIA		
CO5 Statement:	Analyse regular expression patterns. (BL-3)		
Student Name:	A. Mohith krishna	Reg. No.:	192425199
Section / Batch:	AI&ML	Date:	

Code of Conduct

I (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Instructions

- Show all intermediate steps, calculations, state transitions, and reasoning wherever applicable.
- Use only the Python re (Regular Expressions) module to solve the given problems.
- Clearly mention the Regular Expression (Regex) pattern used before writing the corresponding Python program.
- Display the required output for each question, including extracted values, validation results, counts, and positions wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Generation	Pattern Matching Information Extraction	1
Search for a String	Keyword Search and Text Analysis	2
Split the documentation into sentences and words	Text Tokenization and Sentence Segmentation	3
Email and Strong Password Validation	Data Validation and Format Verification	4
Compile Once, Validate Many	Pattern Reusability and Performance Optimization	5

Section 1: Regular Expression Pattern Generation

Student Details:

Name: John Doe

Email: john.doe123@gmail.com

Mobile: +91-9876543210

Password: P@ssw0rd123

Date of Birth: 15/08/2004

Register Number: 23AIML1056

Department: Artificial Intelligence and Machine Learning

For the given student details formulate Regex patterns for the following:

Email ID

Mobile Number

Strong Password

Date of Birth

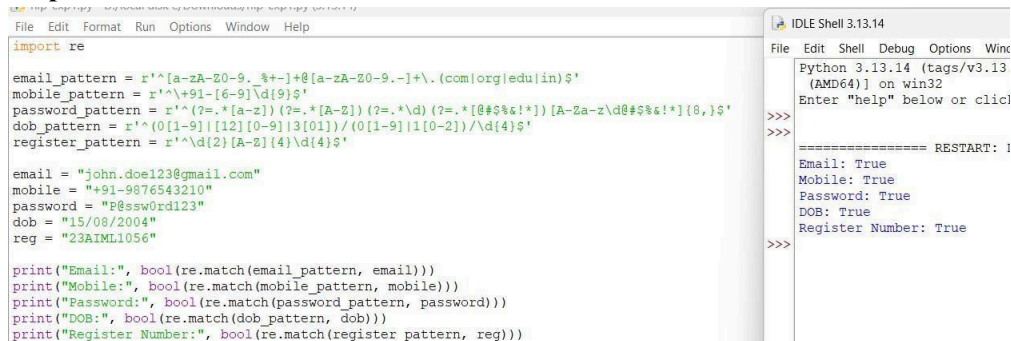
(DD/MM/YYYY) Register

A.Code:

```
import re
email_pattern = ^[A-Za-z0-9._%+-]+@
[A-Za-z0-9.-]+\..com$
mobile_pattern = ^\+91-[6-9]\d{9}$
password_pattern = r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@#%&!*])[A-
Za-z\d@#%&!*]{8,}$'
dob_pattern =
r'^(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])\d{4}$'
register_pattern =
r'^\d{2}[A-Z]{4}\d{4}$'
email =
"john.doe123@gmail.com"
mobile =
"+91-9876543210"
password = "P@ssw0rd123"
dob = "15/08/2004"
reg = "23AIML1056"

print("Email:", bool(re.match(email_pattern, email)))
print("Mobile:", bool(re.match(mobile_pattern, mobile)))
print("Password:", bool(re.match(password_pattern,
password)))
print("DOB:", bool(re.match(dob_pattern, dob)))
print("Register Number:", bool(re.match(register_pattern, reg)))
```

output:



```
File Edit Shell Debug Options Window Help
import re
email_pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\..(com|org|edu|in)$'
mobile_pattern = r'^\+91-[6-9]\d{9}$'
password_pattern = r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@#%&!*])[A-Za-z\d@#%&!*]{8,}$'
dob_pattern = r'^(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])\d{4}$'
register_pattern = r'^\d{2}[A-Z]{4}\d{4}$'

email = "john.doe123@gmail.com"
mobile = "+91-9876543210"
password = "P@ssw0rd123"
dob = "15/08/2004"
reg = "23AIML1056"

print("Email:", bool(re.match(email_pattern, email)))
print("Mobile:", bool(re.match(mobile_pattern, mobile)))
print("Password:", bool(re.match(password_pattern, password)))
print("DOB:", bool(re.match(dob_pattern, dob)))
print("Register Number:", bool(re.match(register_pattern, reg)))

===== RESTART: 1
Email: True
Mobile: True
Password: True
DOB: True
Register Number: True
```

Section 2: Search for a String

Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand.

Write a Python program using the re module to perform the following operations for the word "AI":

1. Find the first occurrence of the word "AI" using re.search().
2. Display the starting position of the first occurrence.
3. Display the ending position of the first occurrence.
4. Display the total number of times the word "AI" appears in the passage. Print an appropriate message if the word is not found.

A) Code:

```
import re
```

```
text = """Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand."""
```

```
pattern = r'AI'
```

```
# 1. Find the first occurrence
result = re.search(pattern, text)
```

```
# 2, 3, 4, 5
```

```
if result:
```

```
    print("First Occurrence :", result.group())
    print("Starting Position :", result.start())
    print("Ending Position :", result.end())
    print("Total Occurrences :", len(re.findall(pattern, text)))
```

```
else:
```

```
    print("Word 'AI' not found.")
```

output:

main.py	   Share	Run	Output
<pre>1 import re 2 3 text = """Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand.""" 4 5 pattern = r'AI' 6 7 # 1. Find the first occurrence 8 result = re.search(pattern, text) 9 10 # 2, 3, 4, 5 11 if result: 12 print("First Occurrence :", result.group()) 13 print("Starting Position :", result.start()) 14 print("Ending Position :", result.end()) 15 print("Total Occurrences :", len(re.findall(pattern, text))) 16 else: 17 print("Word 'AI' not found.")</pre>			<pre>First Occurrence : AI Starting Position : 25 Ending Position : 27 Total Occurrences : 6 === Code Execution Successful ===</pre>

Refer the previous passage and Write a Python program using the `re.split()` method to perform the following operations:

- A.code**

```
sentences = re.split(r'[!?\]+' , text)
sentences = [s.strip() for s in sentences if s.strip()]
print("Sentences:")
for s in sentences:
    print(s)
print("Total Sentences:", len(sentences))
words = re.split(r'\s+', text.strip())
print("\nWords:")
print(words)
print("Total Words:", len(words))
```

```
main.py Run Output Clear
1 import re
2
3 text = """Artificial Intelligence (AI) is transforming industries across the world. AI is used in
healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide
personalized learning experiences. Many companies invest heavily in AI research because AI
improves efficiency and enables intelligent decision-making. As AI continues to evolve,
professionals with AI skills are in high demand."""
4 sentences = re.split(r"[.!?]", text)
5 sentences = [s.strip() for s in sentences if s.strip()]
6 print("Sentences:")
7 print(sentences)
8 print("Total Sentences:", len(sentences))
9 words = re.split(r'\s+', text)
10 print("\nWords:")
11 print(words)
12 print("Total Words:", len(words))
```

Sentences:
['Artificial Intelligence (AI) is transforming industries across the world.', 'AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences.', 'Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making.', 'As AI continues to evolve, professionals with AI skills are in high demand']
Total Sentences: 4

Words:
['Artificial', 'Intelligence', '(AI)', 'is', 'transforming', 'industries', 'across', 'the', 'world.', 'AI', 'is', 'used', 'in', 'healthcare', 'to', 'assist', 'doctors', 'in', 'diagnosis.', 'in', 'banking', 'to', 'detect', 'fraud,', 'and', 'in', 'education', 'to', 'provide', 'personalized', 'learning', 'experiences', 'Many', 'companies', 'invest', 'heavily', 'in', 'AI', 'research', 'because', 'AI', 'improves', 'efficiency', 'and', 'enables', 'intelligent', 'decision-making.', 'As', 'AI', 'continues', 'to', 'evolve.', 'professionals', 'with', 'AI', 'skills', 'are', 'in', 'high', 'demand.']
Total Words: 60

=== Code Execution Successful ===

Section 4: Email and Strong Password Validation

A website requires users to register by providing an Email ID and a Strong Password. Write a Python program using the re module to validate the following inputs.

Validation Criteria:

Email ID

- Must begin with one or more letters or digits.
- May contain ., _, %, +, or - before the @ symbol.
- Must contain a valid domain name.
- Must end with a valid domain extension such as .com, .org, .edu, or .in.

Strong Password

- Must contain at least 8 characters.
 - Must contain at least one uppercase letter.
 - Must contain at least one lowercase letter.
 - Must contain at least one digit.
 - Must contain at least one special character from @, #, \$, %, &, !, or *.
- Must not contain any whitespace characters.

A. Code:

```
import re
email = input("Enter Email: ")
password = input("Enter Password: ")

email_pattern = r'^[A-Za-z0-9]+[A-Za-z0-9._%+]*@[A-Za-z0-9.-]+\.(com|org|edu|in)$'
password_pattern = r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@#$$%&!*])\S{8,}$'

if re.fullmatch(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")

if re.fullmatch(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")
```

output:

main.py	Output
<pre>1 import re 2 email = input("Enter Email ID: ") 3 password = input("Enter Password: ") 4 email_pattern = r'^[A-Za-z0-9]+[A-Za-z0-9._%+]*@[A-Za-z0-9.-]+\.(com org edu in)\$' 5 password_pattern = r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@#\$\$%&!*])\S{8,}\$' 6 if re.fullmatch(email_pattern, email): 7 print("Valid Email ID") 8 else: 9 print("Invalid Email ID") 10 if re.fullmatch(password_pattern, password): 11 print("Strong Password") 12 else: 13 print("Weak Password")</pre>	<pre>Enter Email ID: amirinenimohith07@gmail.com Enter Password: Amohith@1236 Valid Email ID Strong Password === Code Execution Successful ===</pre>

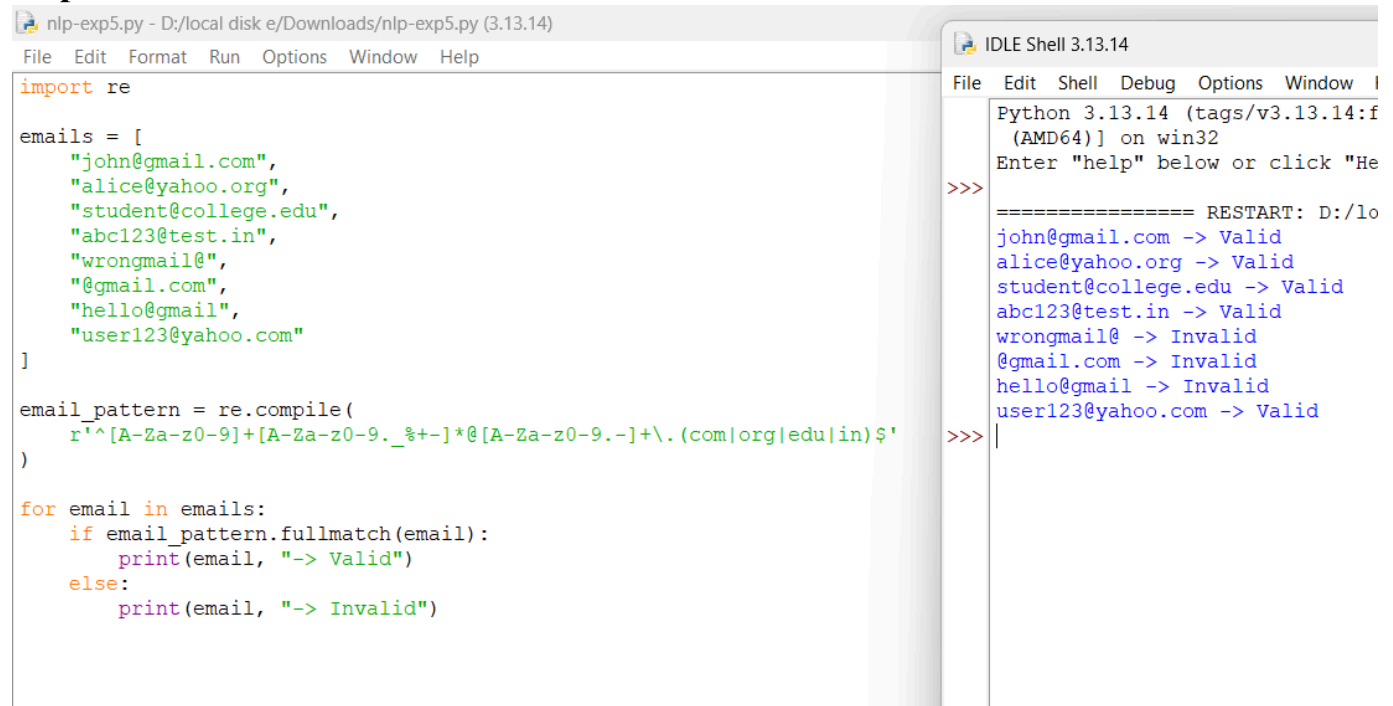
Section 5: Compile Once, Validate Many

A company receives 1,000 email IDs from its customers and needs to validate each one efficiently. Instead of compiling the same regular expression for every email, write a Python program using the `re.compile()` method to compile the pattern once and reuse it to validate all the email IDs.

A. Code:

```
import re
emails = [
    "john@gmail.com",
    "alice@yahoo.org",
    "student@college.edu",
    "abc123@test.in",
    "wrongmail@",
    "@gmail.com",
    "hello@gmail",
    "user123@yahoo.com"
]
email_pattern = re.compile(
    r'^[A-Za-z0-9]+[A-Za-z0-9._%+]*@[A-Za-z0-9.-]+\.(com|org|edu|in)$'
)
for email in emails:
    if email_pattern.fullmatch(email):
        print(email, "-> Valid")
    else:
        print(email, "-> Invalid")
```

output:



The screenshot shows a Python IDE with two windows. The left window, titled 'nlp-exp5.py - D:/local disk e/Downloads/nlp-exp5.py (3.13.14)', contains the Python code from the previous block. The right window, titled 'IDLE Shell 3.13.14', shows the output of the program. The output lists each email address followed by its validation status: 'john@gmail.com -> Valid', 'alice@yahoo.org -> Valid', 'student@college.edu -> Valid', 'abc123@test.in -> Valid', 'wrongmail@ -> Invalid', '@gmail.com -> Invalid', 'hello@gmail -> Invalid', and 'user123@yahoo.com -> Valid'.

```
import re
emails = [
    "john@gmail.com",
    "alice@yahoo.org",
    "student@college.edu",
    "abc123@test.in",
    "wrongmail@",
    "@gmail.com",
    "hello@gmail",
    "user123@yahoo.com"
]
email_pattern = re.compile(
    r'^[A-Za-z0-9]+[A-Za-z0-9._%+]*@[A-Za-z0-9.-]+\.(com|org|edu|in)$'
)
for email in emails:
    if email_pattern.fullmatch(email):
        print(email, "-> Valid")
    else:
        print(email, "-> Invalid")
```

```
Python 3.13.14 (tags/v3.13.14:f
(AMD64)] on win32
Enter "help" below or click "He
>>>
===== RESTART: D:/lo
john@gmail.com -> Valid
alice@yahoo.org -> Valid
student@college.edu -> Valid
abc123@test.in -> Valid
wrongmail@ -> Invalid
@gmail.com -> Invalid
hello@gmail -> Invalid
user123@yahoo.com -> Valid
>>>
```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q. No. / Criteria	Conceptual Understanding	Accuracy of Answer	Application of Knowledge	Completeness of Response	Clarity & Presentation	Sub. Total	Total %
1	3	3	3	4	4		81
2	3	3	3	4	4		
3	3	3	3	4	4		
4	3	3	3	3	3		
5	3	3	3	3	3		

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____